

ClearCall

Speech Enhancement in Noisy Environments

ENSC 429 Final Project

Group4

Team Members:

Prisca Kitsa

Salem Alsaiari

Ali-Aryo Otufat-Shamsi

Mukun Chen

ABSTRACT

Video and voice calls have become a cornerstone in the corporate world. As remote/work from home becomes more popular, increasingly these meetings happen in noisy environments, degrading intelligibility and making meetings less efficient. This project designs and evaluates five speech enhancement methods: a classical DSP baseline, and improved DSP pipeline with speech noise tracking, a pretrained deep learning model (DeepFilterNet3), and two hybrid DSP and DL fused methods. These methods were benchmarked on VoiceBank+DEMAND using SNR improvement, PESQ-WB, and STOI. The results present a defined quality-complexity and a latency tradeoff ratio.

1. INTRODUCTION

1.1 Problem Statement

Video and voice calls have become the default mode of personal and professional communication, carried over platforms such as Zoom, Microsoft Teams, and Discord. The acoustic environment people call from, however, has changed just as much as the tools themselves with remote and hybrid work routing calls through coffee shops, transit stations, coworking spaces, shared apartments, and cars. These are environments where the communication pipeline was initially not designed around. In these settings, the signal a communication application actually receives is not clean speech, but a mixture of the desired speaker's voice and additive interference: $x[n] = s[n] + v[n]$

where $x[n]$ is the signal captured at the microphone, $s[n]$ is the desired speech, and $v[n]$ is the noise component. The enhancement system only ever observes $x[n]$; $s[n]$ is never available at runtime and is used later, for evaluation only.

In practice, the noise component, $v[n]$ spans several distinct categories of interference, each of which degrades communication differently. These range from:

- **Ambient noise:** continuous background sources such as traffic, HVAC systems, and general environmental hum.
- **Transient noise:** brief, high-energy events such as sirens, door slams, or coughing, which can mask entire words regardless of average noise level.
- **Speech-like interference:** competing voices or background conversation that occupies the same spectral and temporal range as the target speaker, making simple frequency-domain separation ineffective. This is a specific instance of the classic "cocktail party problem," where a listener, or an algorithm, must isolate one voice among several acoustically similar sources.

Left unaddressed, these interference types reduce intelligibility, increase listener fatigue, and can cause entire words or phrases to be lost. The problem this project addresses is therefore: given only the noisy signal $x[n]$, produce an enhanced estimate $\hat{s}[n]$ that is perceptually and objectively closer to the clean speech $s[n]$, computed with low enough latency to remain usable in a live, real-time call.

1.2 Motivation and Significance

The demand for solving this problem is already visible in the consumer market, which shows both that the problem is tractable and that current solutions come with real tradeoffs. Krisp removes approximately 90% of background noise across tested noise types on a cross-platform basis [8]. NVIDIA Broadcast achieves comparable suppression, but only on GPUs with dedicated RTX Tensor cores, restricting it to a subset of hardware [9]. Communication platforms also ship built-in suppression, but as closed, non-configurable components. Together, these existing systems confirm that background-noise suppression is both valuable and achievable in real time, but each imposes a constraint such as hardware dependency, platform lock-in, or a lack of transparency that limits when and how it can be used or studied.

This motivates an approach built from first principles using the DSP techniques covered in this course, evaluated against a pretrained deep-learning system rather than assumed to be inferior to it. Classical spectral subtraction [1] and Wiener filtering [2] are well understood, computationally cheap, and fully explainable, but are known to introduce artifacts such as musical noise when suppression is aggressive. Deep-learning approaches such as DeepFilterNet [7] can learn speech and noise structure directly from data and generally achieve higher perceptual quality, at the cost of transparency and, potentially, computational headroom on constrained hardware. Rather than treating these as mutually exclusive design paths, this project treats them as points on a single design space and asks whether a hybrid of the two can offer a better quality-transparency-latency tradeoff than either alone.

This then leads us to the objectives of this project that include to:

1. Design and implement a real-time classical DSP noise-suppression pipeline (STFT-based spectral subtraction + Wiener filtering) capable of running within a live audio with latency of about $\sim 40\text{ms}$
2. Develop and evaluate a deep-learning-based alternative (DeepFilterNet) as a point of comparison against classical DSP approaches.
3. Explore hybrid approaches that combine DSP and DL outputs, aiming to balance DL's audio quality against DSP's speed and predictability.
4. Establish a rigorous, quantitative evaluation methodology - PESQ, STOI, and SNR improvement against a real benchmark dataset (VoiceBank+DEMAND) and a genuine do-nothing baseline; rather than relying on subjective listening alone.
5. Deliver a working end-to-end system that routes enhanced audio through a virtual microphone, so it functions as a drop-in noise-suppressed input for real video-calling applications like Zoom or Teams.

1.3 Background and Literature Review

Classical speech enhancement operates on the same noisy-signal model introduced in Section 1.1: $x[n] = s[n] + v[n]$. The dominant strategy for recovering $s[n]$ from $x[n]$ is to work frame-by-frame in the frequency domain, estimate how much of each frequency bin is likely to be noise, and apply a suppression gain before reconstructing the waveform. The techniques used are described below:

Spectral subtraction, introduced by Boll [1], was one of the earliest practical formulations of this idea. It estimates the noise magnitude spectrum during non-speech intervals and directly subtracts a scaled version of it from the noisy magnitude spectrum, retaining the noisy phase. The method is computationally inexpensive and easy to reason about, but its central weakness as confirmed in our own Original DSP results in Section 5, is that isolated, over-attenuated bins produce audible tonal artifacts commonly called **musical noise**.

Wiener filtering takes a statistical approach by computing a gain based on the estimated a priori signal-to-noise ratio (SNR) in each bin, attenuating bins with low estimated SNR and preserving bins with high estimated SNR [2]. Ephraim and Malah [3] extended this into the **decision-directed a priori SNR estimator**, which blends the previous frame's estimated clean-speech power with the current frame's instantaneous SNR evidence. This smooths gain fluctuations frame-to-frame and reduces musical noise relative to spectral subtraction alone, at the cost of a slower response to sudden changes in the noise floor. This method is used in the Wiener stage of both our Original DSP and Improved DSP methods (Section 2.1.5).

Both spectral subtraction and Wiener filtering depend on accurately tracking the noise floor, which is itself a well-studied subproblem. Martin's minimum-statistics estimator [4] tracks a running minimum of smoothed signal power, under the assumption that the lowest recently observed power in each bin approximates the noise floor; its known weakness is that this tracked minimum can lag for as long as the analysis window, causing slow adaptation when background noise suddenly increases. This is the exact limitation our Original DSP baseline inherits (Section 2.1.3). Cohen's MCRA method [5] addresses this by adding an explicit speech-presence probability to the update rule, allowing the noise estimate to adapt quickly when speech is likely absent and conservatively when speech is likely present, which directly motivates the noise-estimation redesign in our Improved DSP method (Section 2.2.2).

Deep-Learning speech enhancement: Deep-learning approaches instead learn the noisy-to-clean mapping directly from paired training data, implicitly capturing signal's structure that fixed equations cannot easily encode. This project uses DeepFilterNet3 [7] as its comparator: for the lowest ~ 96 bins (0–4.8 kHz), it applies a

multi-frame deep filter, where a network predicts complex coefficients $C[t,i,k]$ across several nearby frames to jointly adjust magnitude and phase; higher-frequency bins use conventional gain masking, $G[t,k]$. This split reflects that most perceptually critical speech structure lives at lower frequencies, where deep filtering's added cost has the greatest payoff. The full pipeline as implemented here appears in Section 2.3. The pretrained model was used as-is, without additional training, keeping the project's contribution focused on DSP design and experimental comparison rather than neural-network development.

DeepFilterNet3 was selected because it provides a pretrained, open-source, CPU-friendly model that natively processes ClearCall's 48 kHz audio and closely matches its DSP frame structure. Other models were not selected because many require substantial training data, operate mainly at 16 kHz, or demand more computation; our dataset is insufficient for training or fine-tuning. DeepFilterNet3 therefore offered the best practical balance of quality, compatibility, computational cost, and implementation effort.

2. Engineering Design and Implementation

2.1 Original DSP Method

Its central idea is to divide the recording into short frames, estimate which frequency components are likely to be noise, reduce those components, and reconstruct the waveform.

The runtime model assumes:

$$x[n]=s[n]+v[n],$$

where $x[n]$ is the noisy microphone signal, $s[n]$ is the unknown clean speech, and $v[n]$ is the noise. The algorithm only receives $x[n]$; clean speech is used later for evaluation, not during enhancement.

The Original DSP architecture can be separated into 7 main parts:

1. **Framing/windowing:** Divide noisy audio into short overlapping frames.
2. **FFT:** Convert each frame from the time domain to the frequency domain.
3. **Noise estimation:** Estimate the background-noise level in each frequency bin.
4. **Spectral subtraction:** Calculate the first suppression gain by subtracting estimated noise.
5. **Wiener filtering:** Calculate the second suppression gain based on estimated SNR.
6. **Gain combination:** Multiply spectral-subtraction and Wiener gains, then apply gain to noisy spectrum.
7. **IFFT and overlap-add:** Convert each processed frame back to the time domain and combine the overlapping frames into the enhanced waveform.

2.1.1. Frame extraction and windowing

Speech changes over time. The complete recording is too long to treat as one stationary signal, but during a short interval such as 20 ms, its frequency content changes less.

Parameter: Value	Parameter: Value	Parameter: Value
Sample rate: 48,000 Hz	Frame length: 960 samples = 20 ms	Hop size: 480 samples = 10 ms
Frame length: 960 samples = 20 ms	Window: Periodic Hann window	Overlap: 50%

A Hann window is applied to reduce spectral leakage caused by abrupt frame boundaries:

$w[n] = 0.5 - 0.5\cos(\frac{2\pi n}{N})$ where $w[n]$ is the periodic Hann-window coefficient and N is the frame length, equal to 960 samples. This makes the frequency analysis smoother. The textbook connects these ideas to windowing.

2.1.2. Real FFT and spectrum analysis

After windowing, the system transforms the frame from time domain to frequency domain:

$X_t[k] = \sum_{n=0}^{N-1} x_t[n]w[n]e^{-j2\pi kn/N}$ Here, t is the frame number; k is the frequency-bin number; $X_t[k]$ is a complex frequency value. Because the input waveform is real, the implementation uses a real FFT, or rfft. A 960-point real FFT produces 481 nonnegative-frequency bins by $K = (960/2) + 1 = 481$.

The frequency spacing is $\Delta f = \frac{F_s}{N} = \frac{48000}{960} = 50\text{Hz}$ where Δf is the frequency spacing between adjacent bins and F_s is the sampling frequency. So, we have Bin 0 = 0 Hz to Bin 480 = 24,000 Hz

Each complex frequency value contains: $X_t[k] = M_t[k]e^{j\Phi_t[k]}$, $M_t[k] = |X_t[k]|$ where $M_t[k]$ is the noisy-spectrum magnitude and $\phi_t[k]$ is its phase. The algorithm changes the magnitude but preserves the original noisy phase.

2.1.3. Minimum-statistics-style noise estimation

This part attempts to estimate the noise magnitude in every frequency bin.

Step 1: Calculate Power

$P_t[k] = |X_t[k]|^2$ where $P_t[k]$ is the noisy-signal power in frame t and bin k .

Step 2: Smooth the Power

The power is then temporally smoothed: $\bar{P}_t[k] = 0.9\bar{P}_{t-1}[k] + 0.1P_t[k]$ where $\bar{P}_t[k]$ is the temporally smoothed power and $\rho=0.9$ is the power-smoothing factor. The previous estimate receives 90% weight, while the current frame receives 10%. This prevents the power estimate from jumping sharply between frames.

Step 3: Find the Recent Minimum

The implementation stores the last 120 smoothed-power frames. Because each frame advances by 10 ms, this represents $120(10\text{ms}) = 1.2\text{s}$

For every frequency bin: $P_{\min,t}[k] = \min_{\tau} \bar{P}_{t-\tau}[k]$ where $P_{\min,t}[k]$ is the minimum smoothed power for frequency bin k , and τ covers the available frames in the recent 1.2-second window.

Step 4: Estimate Noise Magnitude

$\hat{N}_t[k] = \sqrt{bP_{\min,t}[k]}$, $b = 0.5$ where $\hat{N}_t[k]$ is the estimated noise magnitude and b is the empirical noise-power calibration factor. This approach assumes that speech is intermittent while noise is more continuously present. Therefore, the lowest recent power in each frequency bin is treated as an approximation of the noise floor.

Main weakness?

Suppose the background suddenly becomes louder. The buffer may still contain an old low value. That old minimum can remain for as long as 1.2 seconds, so the noise estimate may respond slowly to increasing noise. This is one major weakness that the Improved DSP later addresses in section 2.2.

2.1.4. Spectral Subtraction

Spectral subtraction calculates the first suppression gain by reducing the noisy magnitude according to the estimated noise: $M_{ss,t}[k] = \max(M_t[k] - 0.3\hat{N}_t[k], 0.2M_t[k])$ where $M_{raw,t}[k]$ is the magnitude after direct noise subtraction, $\alpha_{ss}=0.3$ is the oversubtraction factor, $M_{floor,t}[k]$ is the minimum retained magnitude, $\beta=0.2$ is the spectral-floor factor, and $M_{ss,t}[k]$ is the final spectral-subtraction magnitude.

The equivalent gain is $G_{SS,t}[k] = \frac{M_{SS,t}[k]}{M_t[k] + 10^{-12}}$ where $G_{SS,t}[k]$ is the spectral-subtraction keep gain.

The spectral floor prevents this stage alone from completely removing individual frequency bins, which reduces the risk of isolated spectral holes and musical-noise artifacts. However, this floor applies only to the spectral-subtraction gain and does not guarantee a minimum value for the final combined gain.

2.1.5. Decision-Directed Wiener Filtering

The Wiener stage estimates whether each frequency bin contains more speech or more noise.

A Posteriori SNR. This compares the current noisy power with the estimated noise power: $\gamma_t[k] = \frac{|X_t[k]|^2}{\hat{N}_t^2[k] + 10^{-10}}$

where $\gamma_t[k]$ is a posteriori SNR calculated from the current noisy and estimated noise powers.

It describes the current observed SNR.

A Priori SNR. The method then combines the previous frame's estimated clean power and the current frame's instantaneous evidence. $\xi_t[k] = 0.7 \frac{P_{clean,t-1}[k]}{\hat{N}_t^2[k] + 10^{-10}} + 0.3 \max(\gamma_t[k] - 1, 0)$ where $\xi_t[k]$ is the decision-directed

a priori SNR, $a=0.7$ is the Wiener temporal-smoothing factor, and $P_{clean,t-1}[k]$ is the estimated clean-speech power retained from the previous frame. The 0.7 term gives the method memory. This reduces sudden gain changes but can also make it respond slowly.

Wiener Gain $G_{W,t}[k] = \frac{\xi_t[k]}{\xi_t[k] + 1}$ where $G_{W,t}[k]$ is the Wiener keep gain.

A high estimated a priori SNR produces a gain closer to 1, while a low SNR produces stronger attenuation.

The clean-power estimate saved for the next frame is $P_{clean,t}[k] = G_{W,t}^2[k] |X_t[k]|^2$ This is why the Wiener stage must retain state between frames.

2.1.6. Gain Multiplication and Application

The spectral-subtraction gain and Wiener gain are calculated independently from the same noisy spectrum. They are then multiplied to form one final gain: $G_t[k] = G_{SS,t}[k] G_{W,t}[k]$ This combined gain is applied once to the original complex spectrum: $Y_t[k] = G_t[k] X_t[k]$ where $G_{orig,t}[k]$ is the final combined Original DSP gain and $Y_t[k]$ is the enhanced complex spectrum. Because both gains are between 0 and 1, their product can produce stronger attenuation than either gain alone.

2.1.7. Inverse FFT and Overlap-Add

After gain processing, the enhanced frequency spectrum is converted back to a time-domain frame using the inverse real FFT: $y_t[n] = \text{irfft}\{Y_t[k]\}$ Because adjacent 960-sample frames overlap by 50%, the first 480 samples of the current frame are added to the saved second half of the previous frame. The remaining 480 samples are stored for the next frame: $y_{out,t}[n] = y_t[n] + y_{overlap,t-1}[n], 0 \leq n \leq 480$ The Hann window and 50% overlap allow neighboring frames to combine smoothly, reducing discontinuities at frame boundaries. Each 960-sample processed frame therefore produces one 480-sample output block.

2.2 DSP Method Improvement and Engineering Iteration

We call the improved DSP version the Improved DSP, and it keeps the same structure as the Original DSP. However, **three key parts were redesigned**: streaming-compatible block framing, noise estimation, and gain

processing. The Improved DSP was developed through an engineering iteration process with explanation below.

2.2.1. Streaming-Compatible Block Framing

Problem identified

The Original DSP requires a complete 960-sample frame, corresponding to 20 ms of audio. However, the live audio callback provides only 480 samples, or 10 ms, at a time. Therefore, the original processing interface does not directly match the real-time audio input structure.

- Live callback: 480 samples
- Original DSP input: 960 samples

This mismatch means that the Original DSP requires an external adapter before it can be used reliably in the live application.

Engineering Implementation and Iteration

A causal frame adapter was introduced in the Improved DSP. The adapter stores the previous 480-sample block and combines it with the current block: $x_t = \text{concat}(b_{t-1}, b_t)$ where t is the current block index, b_t is the current 480-sample block, b_{t-1} is the previous block, and x_t is the resulting 960-sample analysis frame.

At startup, the previous block is initialized with zeros $b_{-1} = 0$. Therefore, the first analysis frame consists of 480 zero samples followed by the first 480-sample input block.

Engineering rationale

This design was selected because it:

- provides the required 960-sample analysis frame;
- uses only current and past samples;
- does not require future audio;
- preserves the existing 20 ms STFT geometry; and
- produces one 480-sample output block for every 480-sample input block.

The result is a causal real-time implementation with a known one-hop framing delay: $\text{delay} = \frac{480}{48000} = 10\text{ms}$

2.2.2. Noise Estimation Method

Problem identified

The Original DSP estimates noise using the minimum smoothed power found over the previous 120 frames, corresponding to approximately 1.2 seconds: $P_{\min,t}[k] = \min(\text{Previous 120 frames}(\bar{P}_t[k]))$ Here, t is the current frame index, k is the frequency-bin index, $\bar{P}_t[k]$ is the smoothed power, and $P_{\min,t}[k]$ is the minimum smoothed power in bin k over the previous 120 frames.

This method works reasonably well for stationary noise, but it has two important limitations.

- First, when the background noise becomes louder, an older low-power value may remain in the history. As a result, the noise estimate may respond slowly to the new noise level.
- Second, the estimator does not explicitly determine whether speech is present before updating its estimate. Speech energy may therefore influence the estimated noise floor.

Engineering Implementation and Iteration

The sliding-minimum estimator was replaced with an MCRA-style speech-aware noise estimator.

The revised estimator performs four operations: Local frequency smoothing → temporal power smoothing → speech-presence estimation → speech-controlled recursive noise update

The smoothed power is compared with a tracked minimum: $R_t[k] = \frac{S_t[k]}{\max(P_{\min,t}[k], 10^{-12})}$ Here, $S_t[k]$ is the locally and temporally smoothed power, $R_t[k]$ is the smoothed-power-to-minimum ratio, and 10^{-12} prevents division by zero. A binary speech indicator $I_t[k]$ is set to 1 when $R_t[k] > 5$, and to 0 otherwise. It is then smoothed to obtain the speech-presence probability: $p_t[k] = 0.2p_{t-1}[k] + 0.8I_t[k]$ Here, $I_t[k]$ is the current speech indicator, $p_t[k]$ is the current speech-presence probability, and $p_{t-1}[k]$ is the probability from the previous frame. This probability controls the noise-estimator smoothing factor: $A_t = 0.95 + 0.05p_t[k]$ Here, $A_t[k]$ is the speech-controlled smoothing factor. The recursive noise estimate is: $D_t[k] = A_t[k]D_{t-1}[k] + (1 - A_t[k])P_t[k]$ Here, $D_t[k]$ is the current estimated noise power, $D_{t-1}[k]$ is the previous estimate, and $P_t[k]$ is the current noisy power. Before the estimate is used by the gain stages, it is calibrated: $\hat{D}_t = c_D D_t[k] = 0.5D_t[k]$ where $c_D=0.5$ is the empirical noise-power calibration factor. The corresponding estimated noise magnitude is: $\hat{N}_t[k] = \sqrt{\max(\hat{D}_t[k], 0)}$ The calibrated noise power $\hat{D}_t[k]$ is used in the Wiener SNR calculation, while the estimated noise magnitude $\hat{N}_t[k]$ is used by spectral subtraction.

Engineering Rationale

- Update the noise model when speech is absent: When $p_t[k] \approx 0$, the smoothing factor becomes $A_t[k] \approx 0.95$, so about 5% of the current power is used to update the noise estimate.
- Protect speech when speech is present: When $p_t[k] \approx 1$, $A_t[k] \approx 1$, so the noise update slows down or stops. This prevents speech energy from being mistaken for background noise.
- Reduce memory and computation: Instead of storing and scanning a full 120×481 power-history array, the MCRA estimator uses recursive per-frequency-bin states and temporary minimum tracking.

2.2.3. Iteration of Gain Fusion and Post-Processing

Problem Identified

Both methods calculate a spectral-subtraction gain and a Wiener gain. The Original DSP combines them by full multiplication: $G_{orig,t} = G_{SS,t}[k]G_{W,t}[k]$ Here, t is the frame index, k is the frequency-bin index, $G_{SS,t}[k]$ is the spectral-subtraction gain, $G_{W,t}[k]$ is the Wiener gain, and $G_{orig,t}[k]$ is the combined Original DSP gain.

Because both gains attenuate the signal, their multiplication can become too aggressive. For example, gains of 0.6 and 0.4 produce a combined gain of 0.24. In addition, the spectral floor of 0.2 is not a true final floor because multiplication by a small Wiener gain can still reduce the final gain close to zero. This may remove weak speech and create musical-noise artifacts.

Engineering Implementation and Iteration

The Improved DSP introduces three changes:

- 1. Soft Gain Fusion** $G_{F,t}[k] = G_{SS,t}[k](0.5 + 0.5G_{W,t}[k])$ where $G_{F,t}[k]$ is the softly fused gain. This reduces the strength of the Wiener attenuation. Using the previous example, the retained gain increases from 0.24 to 0.42.
- 2. True final gain floor** $G_{B,t}[k] = \text{clip}(G_{F,t}, 0.2, 1)$ where $G_{B,t}[k]$ is the bounded gain. This guarantees that the final attenuation does not reduce a frequency bin below 20%.
- 3. Frequency and temporal smoothing** $0.25G_{B,t}[k-1] + 0.5G_{B,t}[k] + 0.25G_{B,t}[k+1]$ where $G_{Q,t}[k]$ is the frequency-smoothed gain. $G_t[k] = 0.3G_{t-1}[k] + 0.7G_{Q,t}[k]$ where $G_t[k]$ is the final gain and $G_{t-1}[k]$ is its value from the previous frame.

Engineering Rationale

Soft fusion reduces the influence of the Wiener stage instead of always applying its complete attenuation. The final gain floor prevents any frequency bin from being almost completely removed. Frequency smoothing reduces isolated gain peaks and holes between neighboring frequency bins. Temporal smoothing prevents the final gain from changing abruptly between consecutive frames

2.3 Deep Learning Method

DeepFilterNet3 was selected as the pretrained deep-learning method for comparison with the DSP approaches. The noisy signal is modeled as $x[n] = s[n] + z[n]$ where $x[n]$ is the noisy recording, $s[n]$ is the desired speech, and $z[n]$ is the noise.

The model uses a 48 kHz sampling rate, a 20 ms analysis window, and a 10 ms frame step. After the short-time Fourier transform (STFT), the noisy spectrum is represented by $X[t,k]$, where t is the time frame and k is the frequency bin.

For the lowest 96 bins, approximately 0–4.8 kHz, DeepFilterNet3 applies a multi-frame deep filter:

$$Y_{DF}[t, k] = \sum_{i=0}^4 C[t, i, k] X[t + 2 - i, k]$$

Here, deep filter means that the filter coefficients $C[t,i,k]$ are predicted

by a deep neural network. Multi-frame means that five nearby frames are combined to preserve speech patterns over time and adjust both magnitude and phase.

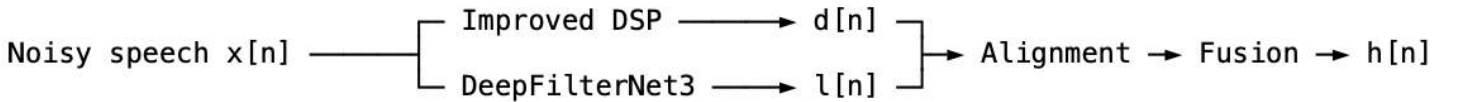
For the remaining frequencies, the model uses gain masking:

$Y_{ERB}[t, k] = G[t, k] X[t, k]$ where $G[t, k]$ controls how much of each frequency component is retained or suppressed. Finally, the enhanced spectrum is converted back into speech using the inverse STFT and overlap-add. The project uses a pretrained DeepFilterNet3 model without additional training.

2.4 Hybrid Method 1

The hybrid methods were proposed to test a specific hypothesis: that combining the complementary strengths of Improved DSP and DeepFilterNet3 could outperform either alone. This is not guaranteed and therefore the hybrid approach is an experimental investigation into whether combining complementary strengths can provide additional benefits, not an assumed improvement.

Hybrid Method 1 combines the outputs of the Improved DSP pipeline and DeepFilterNet3 rather than treating them only as separate competing systems. The noisy speech signal is processed once by each enhancement branch. Their output waveforms are then aligned and fused to produce the final hybrid signal.



Here, $x[n]$ is the noisy input signal, $d[n]$ is the complete Improved DSP output, $l[n]$ is the complete DeepFilterNet3 output, and $h[n]$ is the final hybrid output.

Both branches operate at 48 kHz and produce mono signals with the same number of samples. The reviewed residual delay between the two branches was zero samples, although the alignment stage supports delaying either branch when necessary.

The motivation is that the two methods have different strengths. DeepFilterNet3 provides stronger overall noise suppression, while the DSP branch contributes a transparent and explainable signal-processing path.

Engineering Implementation

2.4.1 Fixed Waveform Fusion

The first part performs direct sample-by-sample blending in the time domain: $h[n] = \alpha l[n] + (1 - \alpha)d[n]$, $0 \leq \alpha \leq 1$ where α is the DeepFilterNet contribution and $1-\alpha$ is the DSP contribution. For example, $\alpha=0.7$ uses 70% DeepFilterNet output and 30% DSP output.

The tested values were $\alpha \in \{0, 0.25, 0.5, 0.7, 1.0\}$ The endpoint values also provide useful references: $\alpha=0$ returns the DSP output, while $\alpha=1$ returns the DeepFilterNet output. This version is computationally inexpensive because it requires only alignment and weighted sample addition.

2.4.2 Frequency-Band Fusion

The second part allows the DSP and DeepFilterNet contributions to vary with frequency. Both aligned signals are divided into synchronized 20 ms frames with a 10 ms hop and transformed using a periodic Hann window and the short-time Fourier transform: $D[t, k] = FFT\{a[n]d_t[n]\}$, $L[t, k] = FFT\{a[n]l_t[n]\}$ where t is the frame index, k is the frequency-bin index, $a[n]$ is the Hann window, and $D[t, k]$ and $L[t, k]$ are the DSP and DeepFilterNet spectra. The complex spectral fusion is: $H[t, k] = w[k]L[t, k] + (1 - w[k])D[t, k]$

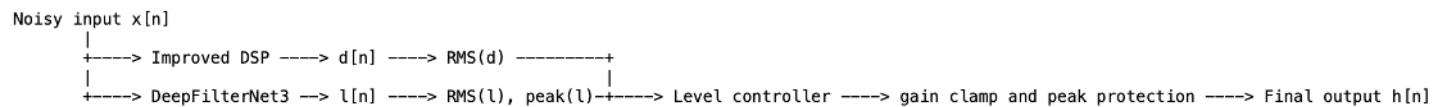
The selected frequency weights were:

Frequency range	DeepFilterNet weight	DSP weight
0–300 Hz	0.25	0.75
300–1000 Hz	0.55	0.45
1000–3000 Hz	0.85	0.15
3000–8000 Hz	0.75	0.25
8000–24000 Hz	0.35	0.65

This design gives DeepFilterNet greater control over the middle speech-frequency region while retaining more DSP contribution at very low and high frequencies. A nine-bin moving average was also tested to smooth abrupt transitions between neighboring frequency bands. In addition to complex-spectrum fusion, magnitude-only fusion was evaluated: $M[t, k] = w[k]|L[t, k]| + (1 - w[k])|D[t, k]|$ The combined magnitude $M[t, k]$ was reconstructed using either the DSP phase or the DeepFilterNet phase. This comparison was included because directly combining complex spectra can cause cancellation when the two branches have different phases. After fusion, the hybrid spectrum is converted back to the time domain using the inverse FFT and weighted overlap-add reconstruction. The output is trimmed to exactly match the original input length.

2.5 Hybrid Method 2

It was designed to use information from Improved DSP without directly mixing its lower-performing waveform into the DeepFilterNet3 output. Improved DSP provides only a reference signal level, while DeepFilterNet3 provides the complete enhanced waveform.



Engineering Implementation

For noisy input $x[n]$, both branches are executed once:

Improved DSP branch: generates $d[n]$, but contributes only its RMS level.

DeepFilterNet3 branch: generates $l[n]$, which supplies the complete final waveform.

Level controller: compares the two RMS levels and applies one bounded scalar gain to $l[n]$.

The RMS level of a signal $z[n]$ is calculated as $RMS(z) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} z^2[n] + \epsilon}$ where N is the number of samples and $\epsilon=10^{-12}$ prevents numerical instability.

The DSP-to-DL level ratio is $r = \frac{RMS(d)}{RMS(l)}$. This ratio is limited to a small upward correction $g_{level} = \text{clip}(r, 1.00, 1.03)$. Thus, the RMS controller cannot reduce the DeepFilterNet3 output and can increase its amplitude by at most 3%. When both branch outputs are near silence, with RMS below 10^{-7} , the gain is set to 1.00.

A peak-safety gain is also calculated: $g_{peak} = \frac{0.999}{\max_n |l[n]|}$ and the applied gain is: $g_{final} = \min(g_{level}, g_{peak})$

The final output is $h[n] = g_{final}l[n]$ where $h[n]$ is the calibrated DeepFilterNet3 waveform. The DSP samples are not added to the output; only their RMS level affects the applied gain.

2.6 Implementation Organization and Real-Time Systems

The enhancement methods were implemented as modular processing pipelines so that each method could be evaluated independently using the same audio-processing framework. The system was organized into components for audio input/output, buffering, signal processing, enhancement, and performance measurement.

The real-time implementation focused specifically on the Original DSP and Improved DSP methods. DeepFilterNet3 and the hybrid methods were evaluated offline and were not integrated into the real-time microphone-to-output system. This separation was necessary because the project focused on demonstrating a computationally efficient DSP-based solution for real-time speech enhancement, while the DeepFilterNet3 and hybrid methods served primarily as offline performance comparisons. For real-time operation, the system receives microphone audio at 48 kHz in blocks of 480 samples, corresponding to 10 ms of audio. Each block is passed through the selected DSP pipeline and then sent to the output. The processing time must remain below the 10 ms block duration to maintain continuous playback. The systems real-time processing structure is: **Microphone** $\rightarrow$ **Input Buffer** $\rightarrow$ **DSP Enhancement** $\rightarrow$ **Output Buffer** $\rightarrow$ **Audio Output**. The Original DSP and Improved DSP maintain internal state between consecutive blocks because their STFT processing uses overlapping frames. The Improved DSP additionally maintains state for its recursive noise estimator and temporal gain smoothing. The block-based implementation allows each new 480-sample input block to be processed while using information from previous blocks without requiring future audio samples.

The real-time system also monitors processing time, real-time factor (RTF), pipeline delay, CPU usage, and audio dropouts. Processing time is measured for individual blocks and compared with the 10 ms processing deadline. An RTF below 1 indicates that the system is processing audio faster than it is being received. Output validation also checks for invalid numerical values, incorrect output length, and excessive signal amplitude. The same DSP implementations were also used in the offline evaluation, allowing the real-time and offline results to be compared using consistent processing methods. The offline experiments additionally included DeepFilterNet3 and the two hybrid methods, which allowed the computationally efficient real-time DSP approaches to be compared against a stronger pretrained deep-learning method and the proposed hybrid approaches.

3. EXPERIMENTAL METHODOLOGY

3.1 Experimental Stages

The experimental evaluation was conducted in several stages to progressively develop, compare, and validate the speech-enhancement methods.

Stage 1 Baseline Development: Original DSP was implemented and tested to identify limitations in noise estimation, gain suppression, and streaming compatibility.

Stage 2 Improved DSP was developed to address the limitations identified in the Original DSP. Deep-learning methods were then explored using DeepFilterNet3, followed by the design and testing of the two hybrid methods to investigate whether DSP and DL could provide complementary benefits.

Stage 3 Offline Method Comparison: All five methods were evaluated on VoiceBank+DEMAND using SNRi, PESQ-WB, STOI, processing time, and output-safety checks.

Stage 4 Computational Feasibility: Methods were tested with consecutive 480-sample blocks (10 ms at 48 kHz) to measure processing latency and deadline overruns under simulated streaming conditions.

Stage 5 Real-Time Evaluation: The DSP-based real-time system was tested with live noisy audio, evaluating speech quality, latency, CPU usage, pipeline delay, and audio dropouts.

This staged approach allowed the project to evaluate both speech-enhancement quality and practical computational feasibility, while separating the offline comparison of all five methods from the real-time evaluation of the implemented DSP system.

3.2 Dataset

The primary dataset used for evaluation was the VoiceBank+DEMAND dataset, which provides paired clean and noisy speech recordings [10]. It combines VoiceBank speech with realistic environmental noise from the DEMAND database. The paired structure allows objective evaluation using clean-reference metrics such as SNRi, PESQ-WB, and STOI.

Approximately 150 matched clean/noisy pairs were used for development. Each noisy recording was processed by the five methods, while the corresponding clean recording was used only for evaluation. Processing was performed at 48 kHz, with signals resampled to the required rate when computing specific evaluation metrics.

A separate subset of 19 noisy recordings was used for simulated streaming tests. Each recording was divided into 480-sample blocks (10 ms at 48 kHz) and processed sequentially to measure processing time, latency, and deadline overruns. The live real-time pipeline used the DSP-based methods, while DeepFilterNet3 and the hybrid methods were evaluated offline as comparison methods.

3.3 Evaluation Metrics

Category	Metric
Noise Reduction	SNR Improvement (SNRi)
Speech Quality	PESQ-WB
Intelligibility	STOI
Offline Speed	Whole-Clip Processing-Time Ratio

Processing Latency	Median, p95, Maximum
Output Safety	Maximum Peak, Clipping Count, NaN/Inf Count
Processing Delay	Pipeline delay (Software Processing Pipeline)
Perception	Blind Listening Ratings (Real-Time)

3.4 Metrics and Explanation

SNR improvement (SNRi): $SNR(c, z) = 10 \log_{10} \left(\frac{\frac{1}{N} \sum_{n=0}^{N-1} c[n]^2}{\frac{1}{N} \sum_{n=0}^{N-1} (z[n] - c[n])^2 + \epsilon} \right)$, $SNRi = SNR(c, y) - SNR(c, x)$

SNRi measures how much enhancement reduces waveform error relative to the clean speech. It calculates the SNR before and after enhancement and subtracts them; a positive value means the enhanced signal is closer to the clean reference. Here, $c[n]$ is the clean speech, $x[n]$ is the noisy input, and $y[n]$ is the enhanced output. The numerator represents clean-speech power, while $(z[n] - c[n])^2$ represents waveform error. SNRi subtracts the noisy-input SNR from the enhanced-output SNR; therefore, a positive SNRi means the enhancement reduces the error.

PESQ-WB: $PESQ_{raw} = 4.5 - 0.1D - 0.0309A$, $PESQ_{WB} = 0.999 + \frac{4}{1 + e^{-1.4945PESQ_{raw} + 4.6607}}$ Explanation:

PESQ-WB estimates perceived speech quality because waveform error alone does not reflect human hearing. It aligns the clean and enhanced signals, compares their perceptual loudness and frequency content, calculates audible disturbances, and converts them into a MOS-like quality score.

STOI: $d_{jm} = \frac{\Sigma x_0 y_0}{(\|x_0\|_2 + \epsilon)(\|y_0\|_2 + \epsilon)}$, $STOI = \frac{1}{JM} \sum_{j=1}^J \sum_{m=1}^M d_{jm}$ Explanation: d_{jm} is the correlation between clean and

enhanced speech envelopes for segment j and frequency band m . J is the number of short-time segments, and M is the number of frequency bands. STOI averages these correlations, so a value closer to 1 means that more intelligible speech information is preserved. STOI measures speech intelligibility by checking whether important short-time speech patterns are preserved. It compares the clean and enhanced speech envelopes across frequency bands, calculates their correlation, and averages the results; a score closer to 1 indicates better intelligibility.

Whole-Clip Processing-Time Ratio $ProcessingTime\ Ratio = \frac{T_{process}}{T_{audio}}$ The processing-time ratio measures offline computational speed by dividing whole-recording processing time by audio duration. A value below 1 means the completed recording was processed faster than its duration.

Processing latency $L_i = (t_{end,i} - t_{start,i})1000ms$, $\bar{L} = \frac{1}{K} \sum_{i=1}^K L_i$ where $t_{end,i}$ is the ending timestamp of audio

block i , $T_{start,i}$ is the timestamp recorded before block i is passed to pipeline. L_i is the latency of one block, while K is the total number of measured blocks. $\bar{L}$ averages all block latencies. Processing latency checks whether each audio block is completed before its deadline. It times individual processing calls and reports median, p95, and maximum values to reveal occasional slow frames.

Output Safety Peak = $\max(|y[n]|)$, $N_{clip} = \sum_n I(|y[n]| \geq 1)$ Output safety verifies that the enhanced audio is technically valid. It checks for clipping, NaN or infinite values, incorrect length, sample rate, or data format that could distort or break playback. Peak shows the largest signal amplitude, while N_{clip} shows how many samples may be clipped and distorted. Ideally, the peak should remain below 1, and $N = 0$

Blind Listening with Human Rating

Blind listening captures qualities that automatic metrics may miss. Listeners rate randomized outputs without knowing the method, based on naturalness, remaining noise, speech damage, artifacts, and overall preference.

4. RESULTS

4.1 Final Method Results

Table 4-1 summarizes the aggregate offline evaluation results for each method. SNRi, PESQ-WB, STOI, and Processing-Time Ratio reported as mean values across the evaluated dataset, while Maximum Peak represents the highest output peak observed during the evaluation and should remain below 1.0 to avoid clipping.

Method	SNRi (dB)	PESQ-WB	STOI	Processing-Time Ratio	Maximum Peak
Original DSP	1.038	2.280	0.9419	0.00850	0.6745
Improved DSP	1.595	2.581	0.9469	0.01391	0.6424
DeepFilterNet3	10.118	3.384	0.9654	0.01872	0.4983
Hybrid Method 2, level calibrated	10.312	3.384	0.9654	0.02794	0.5117
Hybrid Method 1, alpha 0.7	7.774	3.082	0.9607	0.03290	0.5099

Table 4-1. Offline Final Method Results

- Original DSP is fastest, but lowest overall speech quality. It uses a simple DSP pipeline, so computational cost is low. However, strong gain reduction can suppress speech together with noise. Improved DSP is better than Original DSP on SNRi, PESQ, and STOI, with a small increase in runtime.
- The improved gain-control design reduces excessive suppression and better preserves speech. However, it is still limited compared with DeepFilterNet3 because it relies on manually designed signal-processing rules.
- DeepFilterNet3 is the best standalone method and tied for the best PESQ and STOI. The trained neural network can model complex differences between speech and noise more effectively than fixed DSP rules. This provides much stronger noise reduction while preserving speech, although neural-network inference requires more computation.
- Hybrid Method 2 has the highest SNRi. It uses the DSP output to guide a bounded level adjustment of the DeepFilterNet3 output. This improves waveform-level matching and therefore slightly increases SNRi. However, unchanged PESQ and STOI indicate that it did not demonstrate additional perceptual noise reduction.

- Hybrid Method 1 is better than both DSP-only methods, but worse than DeepFilterNet3 and Method 2. It reintroduces residual noise or speech distortion, reducing the quality of the DeepFilterNet3 result. It is also the slowest remaining method because both complete branches must be executed.
- For safety results, all outputs had the correct length and remained below the clipping threshold.

A frame-based offline simulation was conducted to evaluate whether each enhancement method could process audio quickly enough for a future streaming application. The 19 noisy recordings were divided into consecutive blocks of 480 samples. At a sampling rate of 48 kHz, each block represented 10 ms of audio.

NOTE: These processing-time results were obtained on the laptop used for this experiment and should therefore be interpreted as platform-specific measurements. Different hardware, operating-system conditions, background workloads, and power settings may produce different latency results. The measurements demonstrate computational feasibility on the tested system but do not guarantee identical performance on other devices.

This table presents the offline processing latency of the five methods. Median represents the typical processing time, p95 indicates the processing time below which 95% of the measurements fall, and Maximum represents the worst observed processing time. As shown in Table 4-2, these metrics help evaluate each method's potential for real-time processing. In particular, a method with a Maximum Processing Time below the 10 ms processing deadline is generally more suitable for real-time operation because it can consistently complete processing within the required time. Hybrid Method 2 was not included in this evaluation because its processing structure does not support real-time operation.

Method	Median (ms)	p95 (ms)	Maximum (ms)	Calls Over 10 ms
Original DSP	0.0637	0.0709	0.508	0
Improved DSP	0.1116	0.1194	0.6088	0
DeepFilterNet3	0.4128	0.4349	13.8587	1
Hybrid Method 1	0.6159	0.6518	19.047	1
Hybrid Method 2	N/A	N/A	N/A	N/A

Table 4-2. Offline Processing Latency

4.2 Real-Time Evaluation

A separate real-time evaluation was conducted to assess enhancement quality across different noise levels and to measure computational performance. Six configurations were evaluated: bandpass, highpass, Improved DSP, lowpass, no processing, and Original DSP.

4.2.1 Quality Versus Noise Level

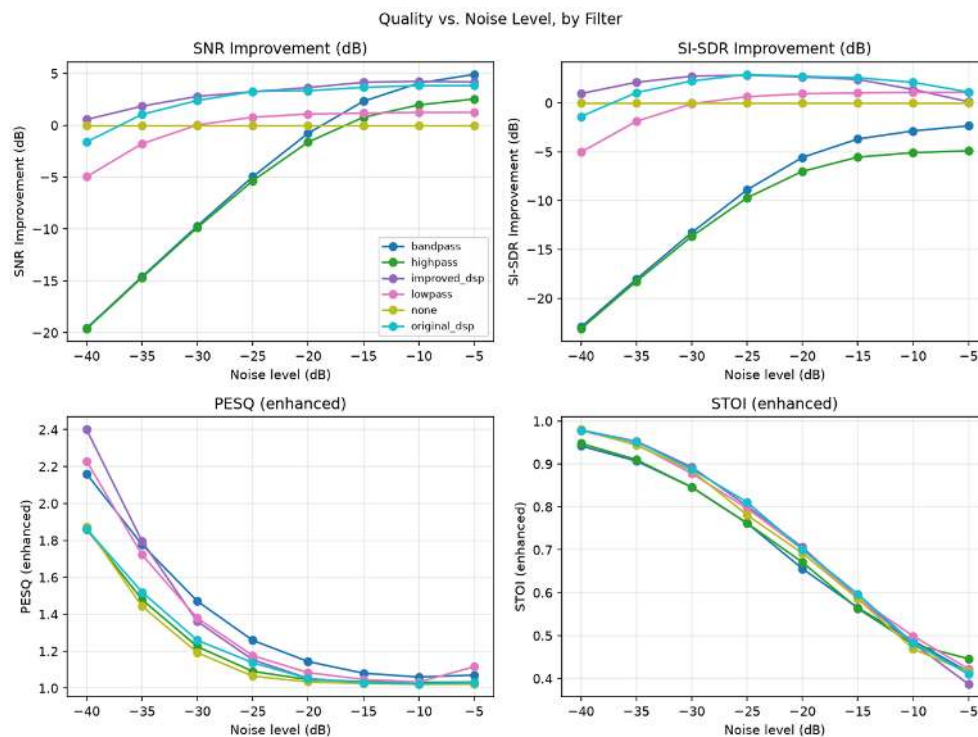


Figure 4-1. Real-time speech-enhancement quality across the noise-level sweep.

Figure 4-1 shows SNR improvement, SI-SDR improvement, PESQ, and STOI across the noise-level sweep. Improved DSP maintained positive SNR and SI-SDR improvement across the full noise-level sweep, while Original DSP became beneficial as the noise level increased. Improved DSP generally provided the strongest performance among the classical filters.

The bandpass and highpass filters produced negative SI-SDR improvement below approximately -15 dB, indicating that they removed speech components along with noise. The lowpass filter performed somewhat better because much of speech energy is concentrated at lower frequencies, but still trailed the dedicated DSP enhancement methods. The no-processing baseline provided approximately zero improvement.

PESQ and STOI decreased for all methods as noise increased. Performance differences between methods were smaller at lighter noise levels, where the noisy input was already closer to clean speech, and there was less opportunity for enhancement methods to differ substantially.

4.2.2 Real-Time Performance

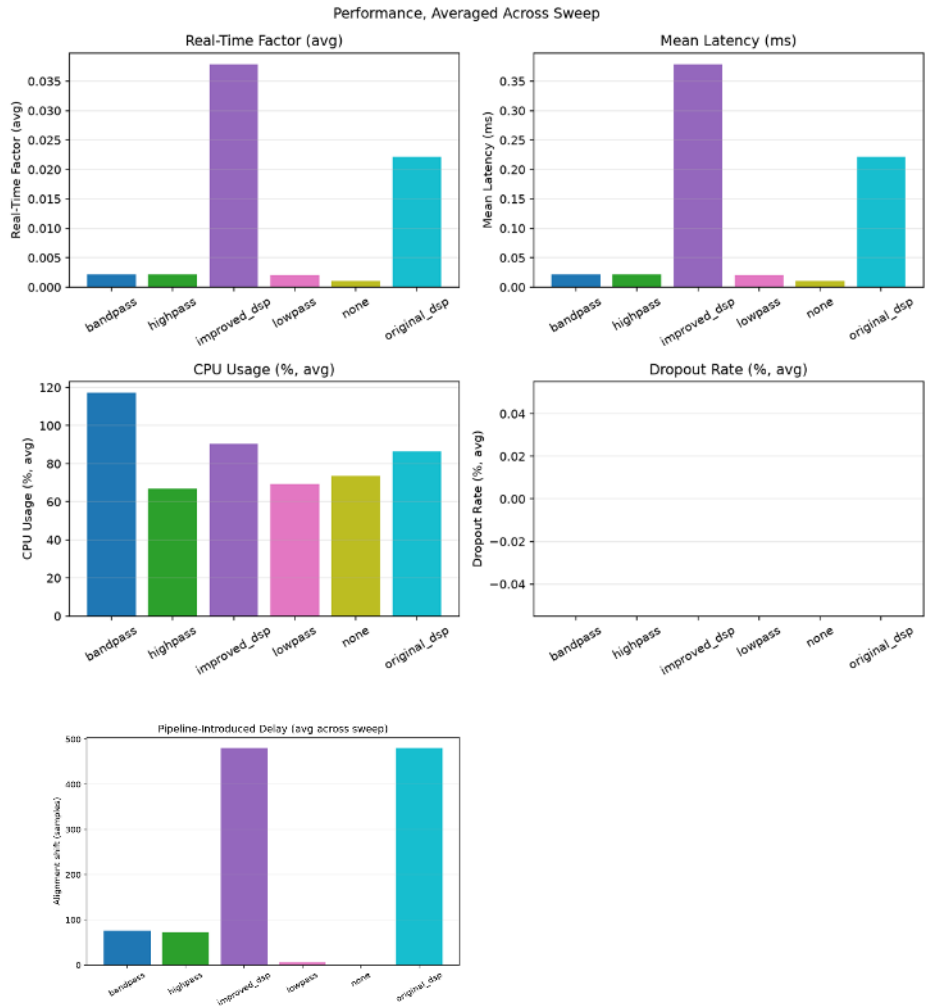


Figure 4-2. Real-time computational performance across the evaluated filters.

Figure 4-2 summarizes RTF, processing latency, CPU usage, pipeline delay, and dropout rate. All six configurations operated with an RTF well below 1.0 and recorded zero dropouts. The highest average RTF was approximately $0.038\times$ for Improved DSP, demonstrating substantial computational headroom.

Improved DSP and Original DSP required more computation than the simpler filters and introduced approximately 480 samples (10 ms) of algorithmic delay at 48 kHz due to STFT frame buffering. Despite this additional delay, both methods maintained continuous real-time operation during testing.

Overall, the real-time evaluation demonstrates that the proposed DSP pipelines provide a useful balance between speech-enhancement quality and computational feasibility, with Improved DSP providing the strongest classical-filter performance at the cost of approximately 10 ms of algorithmic delay.

5. DISCUSSION and ANALYSIS

5.1 Main Findings and Limitations

Original DSP uses spectral subtraction and Wiener filtering, but multiplying both gains can cause excessive suppression. It reduces noise but can also remove weak speech and introduce distortion.

Improved DSP addresses this with speech-aware noise tracking, gentler gain fusion, smoothing, and a gain floor. It generally outperformed Original DSP in SNRi, PESQ, and STOI, but still relies on fixed equations and local signal statistics.

DeepFilterNet3 performed best because it learns speech and noise patterns and uses temporal context. ERB-band gains provide broadband suppression, while deep filtering improves important speech frequencies.

We expected a hybrid approach to combine DeepFilterNet3's denoising with Improved DSP's speech preservation to perform well. Hybrid Method 1 instead performed worse because fixed mixing reintroduced noise and distortion from the weaker DSP branch.

Hybrid Method 2 only adjusts DeepFilterNet3's output level, improving SNRi in many cases. However, because the DSP RMS also contains noise and artifacts, it can amplify unwanted components. Therefore, it is better viewed as output-level calibration rather than improved denoising.

5.2 Comparison: Quality and Latency Tradeoff

Section 4 shows that no single method performs best in every category. Classic DSP has the lowest latency, making it suitable for real-time use, while Improved DSP provides better SNRi, PESQ-WB, and STOI than Original DSP while remaining within the latency budget. DeepFilterNet3 provides much stronger noise suppression and better overall quality, but its latency is 4–7× higher and sometimes exceeds the 10 ms real-time limit. Hybrid Method 1 is even slower and performs worse than standalone DeepFilterNet3, showing that combining methods does not guarantee improvement. Hybrid Method 2 achieved the highest SNRi through output-level adjustment, but its unchanged PESQ-WB and STOI show no additional perceptual or intelligibility benefit. Overall, Improved DSP is the strongest real-time option, while DeepFilterNet3 provides higher quality at the cost of latency.

6. REFLECTION ON THE PROPOSAL PLAN

Below is a summary of changes made since the initial project proposal:

Proposed	Delivered	Why We Adjusted
2 methods: classical DSP vs. DeepFilterNet	5 methods: Original DSP, Improved DSP, DeepFilterNet3, Hybrid Fusion, Hybrid Level Calibration	Once the DSP baseline and DL wrapper worked, we had headroom to test real fusion hypotheses rather than stop at a single comparison.

DNS Challenge dataset	VoiceBank+DEMAND	DNS Challenge was overkill in size/scope for the course timeline; VoiceBank+DEMAND gives matched pairs and relevant noise types at a manageable size.
Cross-platform (Windows + macOS, VB-Cable/BlackHole), Zoom + Teams live test	Windows-only, VB-Cable virtual mic	Narrowing the target platform allowed us to spend the saved integration effort on the DSP engineering iterations and the two hybrid designs instead.
Backup plan: fall back to RNNoise if classical DSP underperforms by Week 4	Not needed since the Improved DSP hit dev-test targets on schedule	The original DSP's known weakness (stale noise floor) was fixed early via MCRA, so the backup trigger was never reached.

Lesson learned:

1. **Improving a well-understood classical method can significantly reduce the performance gap with modern deep-learning methods** while keeping the system simple, transparent, and efficient enough for real-time use. The redesign from Original DSP to Improved DSP (Section 2.2) addressed problems with framing, noise tracking, and gain fusion one at a time. These changes improved SNRi, PESQ-WB, and STOI using only techniques covered in the course. This shows that carefully improving individual parts of a system can sometimes be more effective than making the overall design more complex.
2. **Combining two strong systems does not always produce a better system.** Hybrid Fusion performed worse than DeepFilterNet3 on its own (Section 5.1), which was an important and unexpected result. Instead of assuming that combining DSP and deep learning would improve performance, the project tested this idea directly. Finding that the hybrid approach did not always help was itself a valuable outcome of the project.

7. MEMBER CONTRIBUTIONS

Team members work collaboratively, with each member contributing knowledge from their own research domain.

Member	Role	Technical Task	Deliverable Task
Prisca	DSP Lead	Develop spectral subtraction	Report, presenting slides, presenting
Salem	DL Lead	Implement real-time pipeline	Report, presenting slides, speaker notes
Ali	System Integrator	Perform testing	Report, presenting slides, presenting
Mukun	Quality Lead	Implement the DL filter	Report, presenting slides, presenting

8. CONCLUSION AND FUTURE WORK

8.1 Conclusion

Throughout this project, a goal was set to answer whether a traditional DSP pipeline could compete with professional-grade deep-learning noise suppression models for real-time calls. To take this question to the next level, combinations of classic DSP and deep-learning were used to cover all bases. In total, five unique methods were tested and analyzed. The classic DSP filter methods were shown to have the least latency overall. In the end, the pretrained DeepFilterNet3 deep-learning model has the overall best standalone results. Two combined methods were also tested, combining classic DSP with deep-learning in different ways. The first hybrid method reframed DeepFilterNet3's own output; while the second hybrid method improved waveform level SNRI without changing quality too much. Overall, these results define a solid quality:complexity:latency tradeoff ratio, rather than a single best method.

8.2 Future Work

There are many different directions that remain open beyond this report's scope. Other deep-learning models aside from DeepFilterNet3 could be explored, and experimented against the current model. Furthermore, these openweight models could be fine-tuned, potentially improving results. Deep-learning models can be used within the real-time pipeline, allowing for them to filter out noise in real time during actual meetings. On top of all this, hybrid methods would be changed to support different or fine-tuned models. These hybrid methods would also have to reduce latency for deep learning in real-time systems. On a broader scope, different tooling can be implemented to make these pipelines usable in the real world; updated UI for Zoom, Discord, Slack, Google Meets, Microsoft Teams, etc. Overall, the scope is wide for different directions to take this project for future work.

Source Code - GitHub Repository

<https://github.com/neema66/ClearCall>

REFERENCES

- [1] S. F. Boll, "Suppression of acoustic noise in speech using spectral subtraction," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 27, no. 2, pp. 113–120, 1979.
- [2] J. S. Lim and A. V. Oppenheim, "Enhancement and bandwidth compression of noisy speech," *Proc. IEEE*, vol. 67, no. 12, pp. 1586–1604, 1979.
- [3] Y. Ephraim and D. Malah, "Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 32, no. 6, pp. 1109–1121, 1984.
- [4] R. Martin, "Noise power spectral density estimation based on optimal smoothing and minimum statistics," *IEEE Trans. Speech Audio Process.*, vol. 9, no. 5, pp. 504–512, 2001.
- [5] I. Cohen, "Noise spectrum estimation in adverse environments: Improved minima controlled recursive averaging," *IEEE Trans. Speech Audio Process.*, vol. 11, no. 5, pp. 466 - 475, 2003.
- [6] WebRTC Project, "Noise Suppression Module," webrtc.org. [Online].
- [7] H. Schröter, A. Maier, T. Rosenkranz, et al., "DeepFilterNet: A low complexity speech enhancement framework for full-band audio based on deep filtering," in *Proc. ICASSP*, 2022.
- [8] Krisp Technologies, "Krisp AI Noise Cancellation," krisp.ai. [Online].
- [9] NVIDIA Corporation, "NVIDIA Broadcast," nvidia.com. [Online].
- [10] C. Valentini-Botinhao, "Noisy speech database for training speech enhancement algorithms and TTS models," University of Edinburgh, DataShare, 2017. (VoiceBank+DEMAND)